

Лабораторна робота №1

Тема: Отримання та відображення системної інформації

Виконали: ІП-44 Міліщук Сергій, ІП-44 Комін Іван

Мета роботи

Ознайомлення з базовими принципами функціонування графічної підсистеми Windows та архітектурою програм, керованих подіями. Вивчення життєвого циклу віконного додатка, реєстрації класів вікон, черги повідомлень, а також набуття практичних навичок роботи з Win32 API для отримання параметрів апаратного/програмного середовища та відображення графічної і текстової інформації засобами GDI (Graphics Device Interface).

Завдання до роботи (Варіант 9)

Відповідно до індивідуального варіанта необхідно створити додаток Win32 із наступними параметрами:

- **Стиль класу вікна:** заборонити команду «Закрити» у системному меню (CS_NOCLOSE), увімкнути надсилення повідомлень віконній процедурі при подвійному натисканні миші (CS_DBLCLKS).
- **Тип іконки:** IDI_ERROR (системна піктограма помилки).
- **Тип курсору:** IDC_CROSS (перехрестя).
- **Колір фону:** темно-сірий (DKGRAY_BRUSH).
- **Ім'я класу вікна:** прізвище/ім'я студента англійською мовою.
- **Стиль вікна:** стандартний перекриваний віконний стиль WS_OVERLAPPEDWINDOW (заголовок, системне меню, рамка змінних розмірів, кнопки згортання та розгортання).
- **Початкова позиція та розмір:** значення за замовчуванням (CW_USEDEFAULT).
- **Системні метрики для виведення:**
 - а) Ширина та висота іконки додатку (SM_CXICON, SM_CYICON);
 - б) Ширина та висота вертикальної смуги прокрутки (SM_CXVSCROLL, SM_CYVSCROLL).

Хід виконання роботи

1. Архітектура та реєстрація класу вікна

У точці входу wWinMain виконується ініціалізація структури WNDCLASSEXW у функції MyRegisterClass:

- **Стилі класу:** Для реалізації вимог варіанта полю style присвоєно бітову комбінацію CS_NOCLOSE | CS_DBLCLKS. Прапорець CS_NOCLOSE відключає команду «Close» у системному меню та робить неактивною кнопку закриття вікна (хрестик), а CS_DBLCLKS дозволяє віконній процедурі приймати повідомлення WM_LBUTTONDOWN при подвійному кліку в межах вікна.

- **Візуальні атрибути:** Встановлено піктограму `IDI_ERROR` за допомогою виклику `LoadIcon(nullptr, IDI_ERROR)` та системний курсор `IDC_CROSS` через `LoadCursor(nullptr, IDC_CROSS)`.
- **Фон вікна:** Використано темно-сіру системну кисть `(HBRUSH)GetStockObject(DKGRAY_BRUSH)` що забезпечує автоматичне зафарбовування фону клієнтської області перед перемальовуванням.
- Клас зареєстровано в підсистемі ОС викликом `RegisterClassExW(&wcex)`.

2. Створення вікна та цикл повідомлень

- Створення вікна реалізовано у функції `InitInstance` за допомогою `CreateWindowW`. Передано стиль `WS_OVERLAPPEDWINDOW`, а в якості координат позиції та розмірів вказано константу `CW_USEDEFAULT`, що делегує вибір стартового положення віконному менеджеру Windows.
- Після успішного створення вікно відображається викликами `ShowWindow` та `UpdateWindow`.
- Вхідний потік повідомлень обробляється стандартним циклом `GetMessage` → `TranslateMessage` → `DispatchMessage`, який перенаправляє події до головної віконної процедури `WndProc`.

3. Отримання метрик та відмальовування в GDI

Основна логіка виведення інформації зосереджена в обробнику події `WM_PAINT`:

- Виклик `BeginPaint` ініціалізує структуру `PAINTSTRUCT` та надає дескриптор контексту пристрою (HDC) для малювання виключно у валідованій клієнтській області.
- Для вимірювання системних елементів інтерфейсу викликано функцію `GetSystemMetrics` з відповідними індексами:
 - `SM_CXICON`, `SM_CYICON` — отримання розмірів піктограми додатку (за замовчуванням 32×32 пікселі для стандартного DPI);
 - `SM_CXVSCROLL`, `SM_CYVSCROLL` — отримання ширини вертикального скролбара та висоти його кнопок зі стрілками.
- Отримані числові значення трансформовано у форматований рядок широких символів (`std::wstring`).
- Оскільки фон вікна встановлено темно-сірим, для збереження контрасту та читабельності виконано виклики:
 - `SetBkMode(hdc, TRANSPARENT)` — скасування зафарбовування фону символів власним кольором;
 - `SetTextColor(hdc, RGB(255, 255, 255))` — встановлення білого кольору шрифту.
- Текст виводиться функцією `DrawTextW` з прапорцями `DT_LEFT | DT_TOP | DT_WORDBREAK` у межах прямокутника клієнтської зони (`RECT`), отриманого функцією `GetClientRect`. Завершується малювання викликом `EndPaint`.

4. Обробка користувацьких подій

- Для перевірки роботи прапорця `CS_DBLCLKS` додано перехоплення повідомлення `WM_LBUTTONDOWNBLCLK`. При подвійному кліку лівою кнопкою миші у клієнтській області викликається модальне діалогове вікно `MessageBoxW` з інформаційним повідомленням про успішну фіксацію події.
- Завершення роботи програми відбувається через пункт меню `IDM_EXIT` або команду закриття, що генерує `WM_DESTROY` і виклик `PostQuitMessage(0)`.

Код програми

Вихідний код проєкту, файли ресурсів та налаштування розміщено у відкритому репозиторії GitHub:

<https://github.com/ivankomin/cpp-labs/tree/main/lab1/lab1>

Результати роботи програми

При запуску додатку спостерігаються такі результати відповідно до індивідуального завдання:

1. Вікно коректно ініціалізується з темно-сірим кольором робочої області.
2. Кнопка закриття вікна в заголовку є неактивною (сірою), а в системному меню відсутній активний пункт «Закрити», що підтверджує коректну дію прапорця `CS_NOCLOSE`.
3. При наведенні курсору миші на клієнтську область курсор набуває форми перехрестя (`IDC_CROSS`).
4. У лівому верхньому кутку та на панелі завдань відображається системна піктограма помилки (`IDI_ERROR`).
5. У лівому верхньому кутку робочої області чітко білим текстом відображаються системні параметри (наприклад, на моніторі Full HD зі 100% масштабуванням):
 - Program icon width: 32 px
 - Program icon height: 32 px
 - Vertical scrollbar width: 17 px
 - Vertical scrollbar button height: 17 px
6. Подвійне клацання лівою кнопкою миші викликає інформаційне діалогове вікно, що демонструє працездатність стилю `CS_DBLCLKS`.

Висновки

Під час виконання лабораторної роботи було детально розглянуто архітектуру низькорівневих Win32 API додатків мовою C++. Досліджено механізм реєстрації структури `WNDCLASSEXW` та вплив прапорців стилю класу вікна на його системну поведінку. Освоєно роботу з контекстом графічного пристрою (HDC), правильне використання пари функцій `BeginPaint` / `EndPaint` під час обробки повідомлення `WM_PAINT`, а також засоби форматowanego виведення тексту `DrawTextW`. За допомогою системної функції `GetSystemMetrics` реалізовано динамічне отримання геометричних характеристик інтерфейсних елементів ОС Windows.